

A2C auf Transportaktien — Update-Report

Stand 7. Oktober 2026

Christopher Voizard

7. Oktober 2026

Einordnung vorab: Alle Ergebnisse dieses Reports stammen aus einem *Nachbau* in einer anderen Umgebung als das Original-Notebook. Die Abweichungen sind in Abschnitt 4 vollständig dokumentiert und betreffen unter anderem die Zahl der Input-Features (334 statt 205) sowie die Bibliotheksversionen. Die Zahlen sind deshalb *nicht* 1:1 mit dem Notebook vergleichbar, sondern als unabhängige Plausibilitätsprüfung zu lesen.

Inhaltsverzeichnis

1 Kennzahlen-Update (Fundamentaldaten)	3
2 A2C-Nachbau auf neuen Daten (Boeing)	4
2.1 Ergebnisse je Agent	4
2.2 Handelsverhalten	4
2.3 Interpretation	4
3 Seed-Erweiterung: alle drei Aktien mit 20 Seeds	5
3.1 Ergebnisse	6
3.2 Interpretation	6
4 Methodik-Abweichungen gegenüber dem Original-Notebook	7
5 A3C: ein zweiter Algorithmus, eine Zerlegung und ein generatives Marktmodell	8
5.1 Die Baseline hat keinen Vorsprung	8
5.2 Zerlegung: welcher Faktor trägt?	8
5.3 Ein generatives Marktmodell statt Rolling Windows	9
5.4 Isolation: der Bootstrap trägt, die zwei Passagiere nicht	9
5.5 Was das Overfitting-Bild zeigt	10
5.6 Einschränkungen	10
5.7 Fazit des A3C-Teils	11
6 Artefakte und Reproduktion	11

1 Kennzahlen-Update (Fundamentaldaten)

Methodik wie im Notebook: P/E = trailing PE, P/S = trailing PS, Log-Return = Summe der Tages-Log>Returns des adjustierten Schlusskurses über zwölf Monate.

Tabelle 1: Fundamentalkennzahlen: Original-Notebook (2022) gegenüber 7. Oktober 2026.

Kennzahl	Boeing (BA)	Airbus (AIR.PA)	Toyota (TM)
P/E 2022	n/a (KeyError)	19,887	10,375
P/E 2026	70,79	25,04	7,91
P/S 2022	1,502	1,458	0,0063
P/S 2026	1,60	1,93	0,68
Log-Return 2022	-0,3025	-0,1451	-0,1486
Log-Return 2026	-0,1558	-0,0945	-0,0743

P/E-Verhältnis. Die Aussage aus dem Notebook wird bestätigt und teils verschärft. Airbus bleibt über der Richtgröße von 15 ($19,9 \rightarrow 25,0$), Toyota klar darunter ($10,4 \rightarrow 7,9$). Boeing liefert erstmals einen Wert: **70,79** – eine extrem hohe Bewertungskennzahl. Das ist eine Aussage über die Bewertung, nicht über den Kurs.

P/S-Verhältnis. Das Update hat einen **Datenfehler im Original-Notebook** aufgedeckt. Der dort für Toyota berichtete Wert von 0,0063 (im Text als „nahe null“ und damit als sehr günstig gewertet) war ein **yfinance-Ausreißer**. Der tatsächliche Wert liegt bei **0,68**: Toyota bleibt der günstigste der drei, aber die Aussage „nahe null“ ist falsch. Boeing steht bei 1,60, Airbus bei 1,93.

Log>Returns. Die Kernaussage hält: Auch im letzten Jahr sind alle drei Aktien negativ (BA $-15,6\%$, AIR $-9,5\%$, TM $-7,4\%$), Buy-and-Hold hätte also auch diesmal verloren. Über den längeren Horizont seit dem Ende der Notebook-Daten (August 2022) stehen jedoch alle drei klar im Plus (BA $+20,5\%$, TM $+39,5\%$, AIR $+50$ bis $+65\%$ über fünf Jahre). Das Verlustbild war damit *fensterspezifisch*.

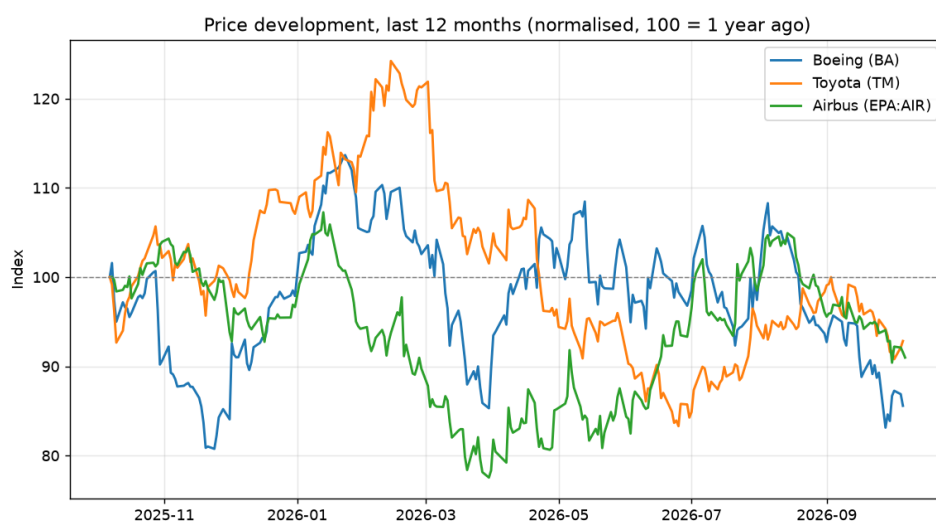


Abbildung 1: Kursentwicklung der drei Aktien über die letzten zwölf Monate (normiert, 100 = vor einem Jahr).

2 A2C-Nachbau auf neuen Daten (Boeing)

Protokoll (aus dem Notebook übernommen): fünf Agenten mit dem Algorithmus A2C (Policy MlpPolicy), 75.000 Trainingsschritte je Agent, Auswertung mit `ai_trade_performance` (`num_plays=20`). Der Test-Environment übernimmt die Merkmalsliste (`use_variables`) und den Skalierer des Trainings-Environments. Agent und Buy-and-Hold laufen auf denselben zwanzig Episoden-Seeds, der Vergleich ist also fair.

Tabelle 2: Zeitfenster des Nachbaus.

	Zeitraum	Datenpunkte
Trainingsfenster (in-sample)	06.10.2021 – 31.05.2024	ca. 660 Handelstage
Testfenster (out-of-sample)	01.06.2024 – 06.10.2026	ca. 580 Handelstage

2.1 Ergebnisse je Agent

End-NAV je Episode (Start = 1,0), Mittel über zwanzig Episoden pro Agent.

Tabelle 3: 2026-Nachbau auf Boeing: End-NAV, Mittel über zwanzig Episoden.

#	Seed	Training Agent	Training B&H	Test Agent	Test B&H	Agent besser
1	100	1,6705	0,9647	0,5616	0,9475	0 %
2	101	2,4073	0,9671	1,2403	0,9408	100 %
3	102	4,0534	0,9956	0,9828	0,9412	75 %
4	103	7,0580	0,9711	1,0106	0,9493	70 %
5	104	5,1547	0,9727	1,1326	0,9485	80 %
∅		4,0688	0,9742	0,9856	0,9455	65 %

2.2 Handelsverhalten

Tabelle 4: Aktionsverteilung der Agenten (je eine Episode).

#	Training short	Training cash	Training long	Test short	Test cash	Test long
1	268	18	348	239	17	299
2	295	39	300	304	24	227
3	297	24	313	279	29	247
4	246	212	176	260	201	94
5	296	15	323	309	12	234

Alle Agenten handeln überwiegend long; nur Agent 4 hält eine große Cash-Position. Short-Positionen werden fast immer gehalten, aber selten stark gewichtet.

2.3 Interpretation

In-sample ist kein Beleg für Können. Im Trainingsfenster erreichen die Agenten im Mittel einen NAV von 4,07 gegenüber 0,97 für Buy-and-Hold und schlagen die Benchmark in *jeder* Episode. Das ist kein Ergebnis: Hier wurde trainiert und hier wurde gemessen, die Zahl zeigt nur, dass die Agenten das Fenster auswendig gelernt haben.

Out-of-sample ist das Bild nüchtern. Im Testfenster liegt der Mittelwert bei 0,9856 gegenüber 0,9455 für Buy-and-Hold, der Agent ist in **65 %** der Episoden besser – aber mit sehr großer Streuung: Agent 1 verliert deutlich (0,56), Agent 2 gewinnt klar (1,24), und zwei von fünf Agenten verlieren in mindestens einem Viertel der Episoden gegen Buy-and-Hold. Der Vorteil liegt innerhalb der Seed-Streuung und ist **nicht signifikant**.

Buy-and-Hold liegt in beiden Fenstern unter 1,0, Boeing hat also in beiden Zeiträumen an Wert verloren. Die Frage lautet hier „weniger verlieren“ statt „gewinnen“. Insgesamt **stützt** der Nachbau die Konklusion des Original-Notebooks: A2C liefert keine verlässliche Outperformance.

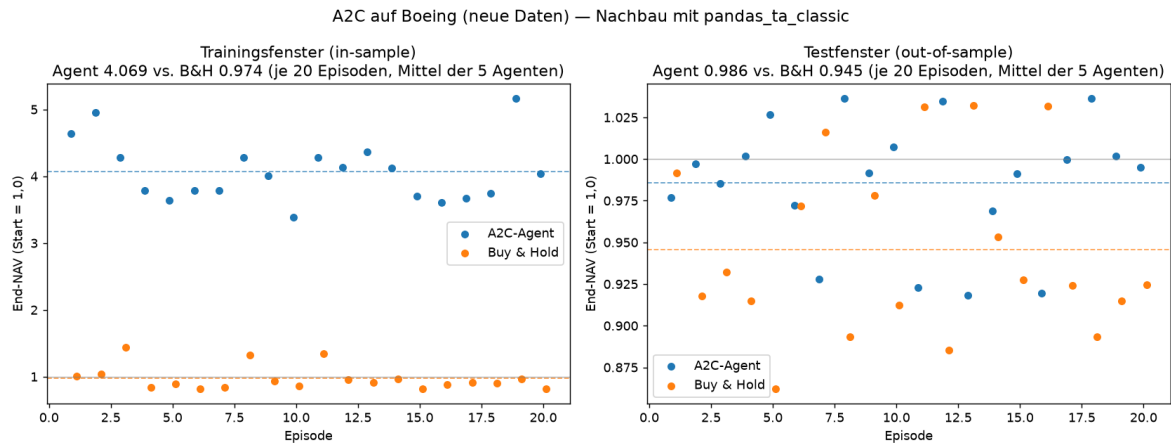


Abbildung 2: End-NAV je Episode (Mittel der fünf Agenten), links Trainingsfenster (in-sample), rechts Testfenster (out-of-sample).

3 Seed-Erweiterung: alle drei Aktien mit 20 Seeds

Warum überhaupt mehr Seeds? Abschnitt 2 kommt auf fünf Agenten (Seeds 100–104). Damit lag der Testvorteil auf Boeing bei +0,04 NAV-Punkten — kleiner als die Streuung zwischen den Agenten. Aus einer so kleinen Stichprobe lässt sich nicht entscheiden, ob das ein Vorteil oder Rauschen ist. Deshalb wurde das Protokoll unverändert auf **20 Seeds (100–119)** erweitert und gleichzeitig auf **alle drei Aktien** angewandt, einschließlich Airbus auf den neuen Daten. Die fünf Agenten aus Abschnitt 2 sind eine Teilmenge dieser zwanzig.

Trainings- und Testfenster sind für alle drei Aktien identisch mit Abschnitt 2 (06.10.2021–31.05.2024 bzw. 01.06.2024–06.10.2026). Verglichen wird jeweils gegen Buy-and-Hold auf *den-selben* zwanzig Episoden-Seeds. Die *Marge* ist die mittlere gepaarte Differenz Agent – Buy-and-Hold über die Seeds; deren Streuung liefert das 95-

3.1 Ergebnisse

Tabelle 5: 2026-Nachbau, 20 Seeds je Aktie. End-NAV (Start = 1,0), Mittel über die Seeds; p aus dem gepaarten t -Test über die Seed-Differenzen (H_0 : Marge = 0); Trefferquote über alle 400 Episoden je Aktie und Fenster.

Aktie	Fenster	Agent	Streuung	95-%-KI	Buy & Hold	Marge	p
Boeing	Training	3,669	1,400	[3,01; 4,33]	0,973	+2,697	< 0,0001
Boeing	Test	1,094	0,231	[0,99; 1,20]	0,943	+0,151	0,0037
Airbus	Training	1,818	0,423	[1,62; 2,02]	1,292	+0,526	< 0,0001
Airbus	Test	1,096	0,156	[1,02; 1,17]	1,091	+0,005	0,887
Toyota	Training	2,455	0,724	[2,12; 2,79]	1,594	+0,861	< 0,0001
Toyota	Test	0,852	0,144	[0,79; 0,92]	0,983	-0,130	0,0001

Tabelle 6: Trefferquoten (Anteil der Episoden, in denen der Agent Buy-and-Hold schlägt) sowie Rendite/Risiko-Verhältnis im Testfenster.

Aktie	Trefferquote Training	Trefferquote Test	Rendite/Risiko Test (Agent vs. B&H)
Boeing	100,0 %	75,0 %	0,299 vs. -0,160 ($p = 0,049$)
Airbus	79,8 %	47,8 %	0,146 vs. 0,322 ($p = 0,22$)
Toyota	87,0 %	13,8 %	-1,038 vs. 0,404 ($p < 0,0001$)

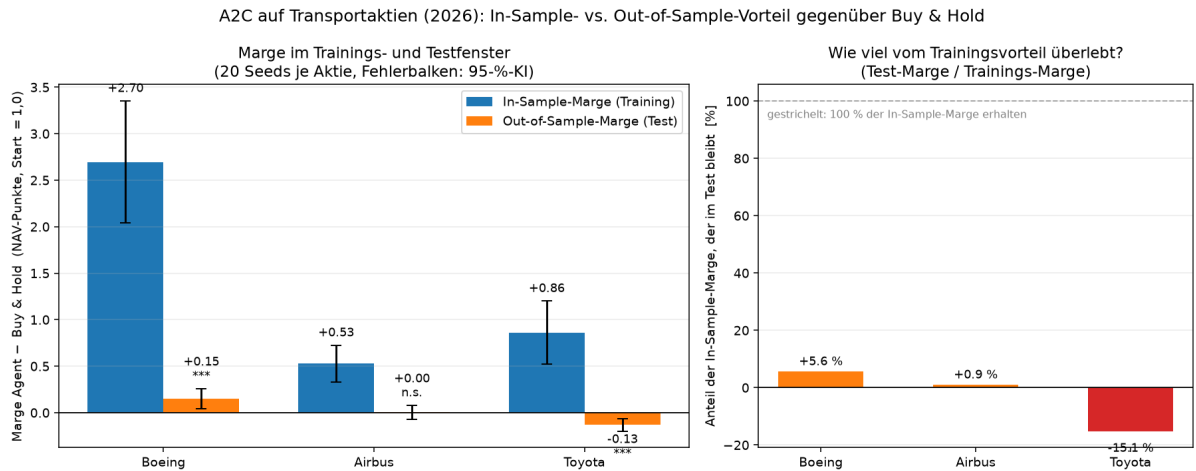


Abbildung 3: Marge (Agent – Buy-and-Hold) im Trainings- und im Testfenster je Aktie. Fehlerbalken: 95-%-Konfidenzintervall der mittleren gepaarten Differenz über 20 Seeds. Rechts: Anteil der In-Sample-Marge, der out-of-sample übrig bleibt.

3.2 Interpretation

In-sample schlägt der Agent überall deutlich. Zwischen +0,53 und +2,70 NAV-Punkten, in allen drei Fällen mit $p < 0,0001$ und Trefferquoten von 80 bis 100 %. Das ist erwartbar und *kein* Beleg für Können: Hier wurde trainiert und hier wurde gemessen.

Out-of-sample bleibt fast nichts übrig. Boeing behält einen signifikanten Vorteil (+0,151, $p = 0,0037$), Airbus landet exakt auf Buy-and-Hold (+0,005, $p = 0,89$, Trefferquote 47,8 % — ein Münzwurf), und Toyota dreht ins Gegenteil (-0,130, $p = 0,0001$, Trefferquote 13,8 %).

Die In-Sample-Marge sagt den Out-of-Sample-Erfolg nicht vorher. Toyota war im Training der zweitbeste und im Test der schlechteste, Airbus im Training der schwächste und im Test der zweitbeste; die Rangfolge der beiden Fenster hat nichts miteinander zu tun. Anschaulich rechts in Abbildung 3: von der Trainingsmarge bleiben bei Boeing 5,6 %, bei Airbus 0,9 % und bei Toyota −15,1 %.

Die Zahl der Seeds hat ein Urteil gedreht. Mit fünf Seeds war der Boeing-Testvorteil (+0,040, Trefferquote 65 %) innerhalb der Streuung und damit nicht signifikant; mit zwanzig Seeds sind es +0,151 bei $p = 0,0037$. Dasselbe Experiment, dasselbe Protokoll — nur eine belastbare Stichprobe.

Gegenüber dem Original-Notebook (2013–2017) war dort für alle drei Aktien ein Agentenvorteil messbar (Toyota 1,27 gegen 1,00 bei 80 % Trefferquote). Auf den neuen Daten ist bei Toyota nichts davon übrig — es kippt ins signifikant Schlechtere. Das **stützt** die Grundaussage des Notebooks, dass A2C keine verlässliche Outperformance liefert, und schärft sie: Der Nachbau findet keine Aktie, bei der der Vorteil robust groß wäre.

Einschränkungen. Zwanzig Seeds sind deutlich besser als fünf, aber weiterhin eine kleine Stichprobe; die Konfidenzintervalle in Tabelle 5 sind entsprechend breit. Die Airbus-Kursreihe stammt aus einer anderen Quelle als Boeing und Toyota (Yahoo, `AIR.PA`, in Euro, 1.307 Handelstage bis 06.10.2026). Und es gilt weiterhin die Haupteinschränkung aus Abschnitt 4: der Beobachtungsvektor ist mit **335** Merkmalen größer als im Original (205), ein Teil der Differenz kann daher stammen.

4 Methodik-Abweichungen gegenüber dem Original-Notebook

Der Nachbau lief *nicht* in der ursprünglichen conda-Umgebung (Python 3.8.8, stable-baselines3 1.6.0, gym 0.21, torch 1.12.1), sondern in einem Linux-Container mit aktuellem Stack. Alle Ersetzungen sind bewusst und dokumentiert.

Tabelle 7: Ersetzungen im Nachbau.

Aspekt	Original-Notebook	Nachbau 2026
Kursdaten	<code>yfinance</code> (live)	lokale CSV
Technische Indikatoren	<code>pandas_ta</code> 0.3.14b0	<code>pandas_ta_classic</code>
Zahl der Indikatoren	204	348
Beobachtungsvektor	205	334
Parallelisierung	Multiprocessing-Pool	seriell
RL-Stack	SB3 1.6.0 + gym 0.21	SB3 2.9 + gymnasium 1.4

Die wichtigste Einschränkung ist der Beobachtungsvektor von 334 statt 205. Der Agent sieht mehr Input-Features als im Original, ein Teil der Leistungsdifferenz kann also daher stammen und *nicht* vom Datenupdate. Hinzu kommt: `yfinance` ist von diesem Host hart geblockt (HTTP 429 auf allen Endpunkten), weshalb die Kursreihe aus einer lokalen Datei gelesen wird, und das Helper-Paket `aif_course` ist nicht mehr verfügbar (das Upstream-Repository antwortet mit HTTP 404). Das Environment musste lokal rekonstruiert werden; die genauen Ersetzungen sind im Begleitcode dokumentiert.

Nicht reproduzierbar ist der Original-Lauf mit den vorab trainierten Agenten (`A2C_showcase_agent1..5`). Es lag nur *ein* Agent vor, ein Vergleich „Original-Agent gegen neu trainierter Agent“ war daher nicht möglich.

5 A3C: ein zweiter Algorithmus, eine Zerlegung und ein generatives Marktmodell

Woher die Frage kommt. Original-Notebook und Nachbau (Abschnitt 2) nutzen *A2C*, also synchrones Lernen. Die naheliegende Gegenprobe ist *A3C*: mehrere Worker sammeln parallel Rollouts und schieben ihre Gradienten in ein gemeinsames Netz (Hogwild-Stil, nur der Optimizer-Schritt ist gesperrt). Umgebung, Fenster, Episoden-Seeds und Auswertungsfunktionen bleiben unverändert — damit misst der Vergleich den Lernalgorithmus und nicht das Drumherum. Die Endauswertung läuft wieder über **20 Seeds (100–119)**. Eine Besonderheit vorweg: A3C wurde erst *nach* der Fertigstellung des A2C-Teils gebaut, es gibt also keine Notebook-Vorlage, gegen die man die Zahlen spiegeln könnte.

5.1 Die Baseline hat keinen Vorsprung

Tabelle 8: A3C auf Boeing, 20 Seeds. Marge = Agent – Buy-and-Hold in NAV-Punkten (Start = 1,0); p einseitig aus dem t -Test “Marge > 0”. “R/R” ist das Rendite/Risiko-Verhältnis im Testfenster.

Arm	Training	Test	95-%-KI Test	Treffer Test	p	R/R Test
Baseline	+8,43	+0,019	[−0,06; +0,10]	51,0 %	0,32	−0,24
ent_coef 0,02	+8,37	+0,047	[−0,03; +0,12]	64,0 %	0,10	−0,37
obs_noise 0,01	+8,79	+0,038	[−0,03; +0,11]	57,2 %	0,14	−0,45
nstep 10	+6,71	+0,125	[+0,01; +0,24]	67,8 %	0,019	−0,05
Bündel (alle drei)	+7,69	+0,145	[+0,03; +0,26]	70,2 %	0,007	−0,03
Bündel + Bootstrap	+0,02	+0,218	[+0,09; +0,35]	77,8 %	0,0012	+0,50
nstep 10 + Bootstrap	+0,02	+0,218	[+0,09; +0,35]	72,5 %	0,0010	+0,38

Die Baseline wiederholt das A2C-Ergebnis: Im Testfenster liegt die Marge bei +0,019 NAV-Punkten, der Agent gewinnt in **51,0 %** der Episoden — ein Münzwurf — und der einseitige p -Wert ist 0,32. **Ein A3C-Agent ohne Anpassungen hat keinen belegbaren Vorsprung vor Buy-and-Hold.** Im Trainingsfenster steht gleichzeitig eine Marge von +8,43: Der Agent hat den einen historischen Pfad auswendig gelernt.

5.2 Zerlegung: welcher Faktor trägt?

Ein naheliegender Griff ins Werkzeugregal ist ein Bündel aus drei Änderungen: **nstep** 5 → 10 (n-Schritt>Returns), **ent_coef** 0,01 → 0,02 (Entropie-Bonus) und **obs_noise** 0 → 0,01 (Gauß-Rauschen auf die Beobachtung während des Trainings). Das Bündel hebt die Testmarge auf +0,145 ($p = 0,007$). Nur: *welcher* der drei Faktoren tut das?

Deshalb wurde jeder Faktor einzeln bei ebenfalls 20 Seeds gefahren (Zeilen 2–4 in Tabelle 8). Das Ergebnis ist eindeutig und war für mich überraschend:

nstep ist der Wirkstoff. Allein erreicht **nstep** 10 bereits +0,125 ($p = 0,019$, Trefferquote 67,8 %) — das sind **85 %** der Bündelwirkung. Der gepaarte Test Bündel gegen **nstep** allein ergibt +0,020 bei $p = 0,81$: statistisch **nicht unterscheidbar**. Das Bündel ist also keine Summe von Hebeln, sondern im Wesentlichen **nstep** plus Rauschen.

ent_coef und obs_noise sind inert. Einzeln liefern sie +0,047 ($p = 0,64$) und +0,038 ($p = 0,72$) gegen die Baseline, mit 10 bzw. 9 von 20 positiven Seed-Differenzen — ein Münzwurf. Auch auf das Overfitting wirken sie nicht: Die In-Sample-Marge bleibt bei +8,37 bzw. +8,79 gegenüber +8,43 in der Baseline ($p = 0,90$ und 0,53). Nur **nstep** 10 senkt sie signifikant auf +6,71 ($p = 0,0015$) — und hebt gleichzeitig den Testwert.

Additivität. Die Summe der drei Einzel-Deltas ist +0,155, das Bündel liegt bei +0,126; das Residuum ist −0,029 bei einer Streuung von 0,61 ($p = 0,84$). Der Punktschätzer ist also leicht subadditiv ($0,82\times$), aber von reiner Additivität statistisch nicht zu unterscheiden — das Rauschen dominiert. Eine Synergie gibt es jedenfalls nicht.

5.3 Ein generatives Marktmodell statt Rolling Windows

Die zweite Idee stammt aus der Paper-Auswahl (Kuo et al., LOB-GAN): Generalisiert der Agent besser, wenn er *mehrere plausible Marktpfade* sieht statt eines einzigen historischen? Der nahe-liegende Weg wären rollierende Trainingsfenster — und der ist in diesem Protokoll **nicht bau-bar**. Die Umgebung erzwingt pro Environment mehr als $2 \cdot 252 = 504$ Beobachtungen (`assert len(df) > 2*trading_days`), und `trading_days` steuert gleichzeitig die Episodenlänge von 252 Schritten; das Minimum lässt sich also nicht absenken, ohne das Protokoll zu brechen. Das Trainingsfenster hat aber nur **634** nutzbare Zeilen, und die Kursreihe beginnt exakt am Fensteranfang. Zwei echte Sub-Fenster bräuchten je rund 550 Zeilen — unmöglich. Der Code prüft das inzwischen selbst und bricht mit klarer Meldung ab.

Stattdessen erzeugt ein **Block-Bootstrap** synthetische Kursreihen. Entscheidend ist dabei, *was* resampelt wird: Preisniveaus wären falsch, weil an jeder Blocknaht ein künstlicher Kurssprung entstünde (Block endet bei 100, der nächste beginnt bei 80 — minus 20 % über Nacht). Gebootstrapt wird deshalb die *Tagesform*: $\log(\text{Close}_t/\text{Close}_{t-1})$, $\log(\text{Open}_t/\text{Close}_{t-1})$, $\log(\text{High}_t/\text{Close}_t)$, $\log(\text{Low}_t/\text{Close}_t)$ und $\log(\text{Volume}_t)$. Blöcke von 20 aufeinanderfolgenden Tagen werden mit Zurücklegen gezogen und der Pfad wird vom ersten echten Close aus neu aufgebaut. Innerhalb der Blöcke bleibt die Autokorrelation erhalten, die Kurve ist per Konstruktion stetig, und der Agent sieht den *realen* Trainingspfad nie. Gegenprobe: Die Streuung der Tagesrenditen beträgt in den synthetischen Reihen 0,0229 bzw. 0,0247 gegenüber 0,0239 in der echten Reihe.

5.4 Isolation: der Bootstrap trägt, die zwei Passagiere nicht

In der Zeile “Bündel + Bootstrap” steckt der Bootstrap noch zusammen mit den zwei inerten Faktoren. Der isolierende Lauf `nstep 10 + Bootstrap` ohne `ent_coef` und `obs_noise` schließt die Lücke (letzte Zeile):

Tabelle 9: Gepaarte Tests über die 20 Seed-Differenzen ($df=19$), Testmarge.

Vergleich	Δ Testmarge	t	p
<code>nstep 10</code> – Baseline	+0,107	1,69	0,107
<code>ent_coef 0,02</code> – Baseline	+0,028	0,47	0,643
<code>obs_noise 0,01</code> – Baseline	+0,020	0,36	0,720
Bündel – Baseline	+0,126	1,98	0,062
Bündel – <code>nstep 10</code>	+0,020	0,24	0,810
<code>nstep 10 + Bootstrap</code> – Baseline	+0,200	2,35	0,030
<code>nstep 10 + Bootstrap</code> – <code>nstep 10</code>	+0,093	1,16	0,260
<code>nstep 10 + Bootstrap</code> – Bündel + Bootstrap	+0,0002	0,003	0,998

Der Bootstrap-Effekt hält isoliert — und war nie von den Passagieren getragen. Der isolierte Zuwachs beträgt +0,093 und damit *exakt* denselben Wert wie die kontaminierte Schätzung (+0,093); das Verhältnis ist 1,00. Der Effekt ist also weder geschrumpft noch verschwunden. Einschränkung: Für sich allein ist er **nicht signifikant** ($p = 0,26$, 12 von 20 Seeds positiv, Median +0,138, getrimmter Mittelwert +0,112) — der Punktschätzer ist stabil, die Stichprobe zu klein.

nstep 10 + Bootstrap ist von “Bündel + Bootstrap” nicht zu unterscheiden ($+0,0002$, $p = 0,998$). Damit sind `ent_coef` und `obs_noise` **endgültig als Passagiere belegt**: Die beiden Läufe unterscheiden sich nur in diesen zwei Parametern und liefern praktisch dieselbe Verteilung. (Nebenbei ein Konsistenzbeleg dafür, dass die Bootstrap-Replikate deterministisch sind.)

Erstmals ein signifikanter Vorsprung. `nstep 10 + Bootstrap` ist gegenüber der Baseline signifikant besser ($+0,200$, $p = 0,030$) und die eigene Testmarge ist klar von null verschieden ($p = 0,0010$, zweiseitig $0,0019$). Das ist der *erste* Arm in diesem Projekt, der die 5-%-Schwelle im gepaarten Test gegen die Baseline reißt — weder `nstep` allein ($p = 0,107$) noch das Bündel ($p = 0,062$) schaffen das.

A3C auf Boeing: Ein-Faktor-Zerlegung und Block-Bootstrap (20 Seeds, Seeds 100–119)

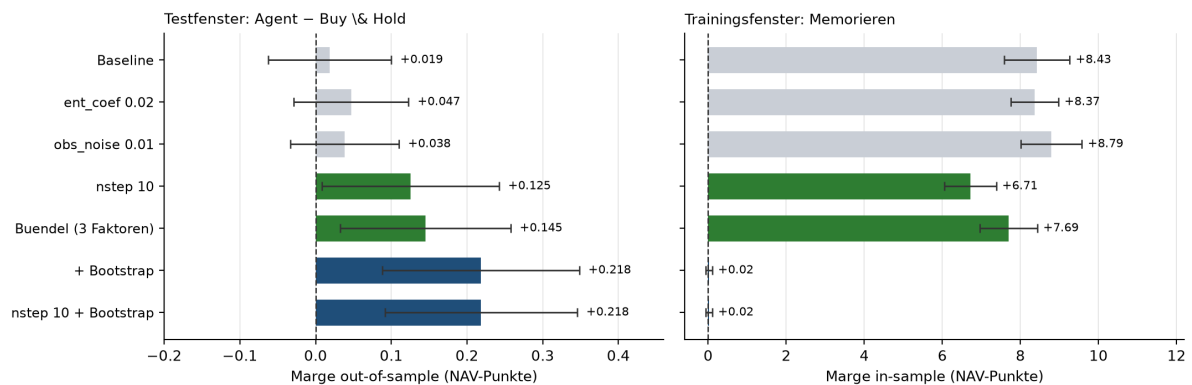


Abbildung 4: Ein-Faktor-Zerlegung und Block-Bootstrap, 20 Seeds. Links die Testmarge (Fehlerbalken: 95-%-KI), rechts die Marge im Trainingsfenster. Die beiden Bootstrap-Arme (dunkel) haben in-sample praktisch null Vorsprung und out-of-sample den größten.

5.5 Was das Overfitting-Bild zeigt

Abbildung 4 macht den Kern sichtbar. Die vier Arme, die auf dem echten Fenster trainieren, tragen in-sample zwischen $+6,7$ und $+8,8$ NAV-Punkte — und davon bleibt out-of-sample fast nichts. Die beiden Bootstrap-Arme haben in-sample $+0,02$, also *null* Vorsprung, und liefern gleichzeitig die besten Testwerte. Sie haben den historischen Pfad nie gesehen; dass sie im Trainingsfenster nicht gewinnen, ist deshalb keine Schwäche, sondern der Mechanismus. Ein In-Sample-Vorsprung von $+8$ ist damit als das entlarvt, was er ist: Memorieren.

5.6 Einschränkungen

20 Seeds sind eine kleine Stichprobe. Die Konfidenzintervalle in Tabelle 8 sind breit, und der zentrale Bootstrap-Zuwachs ist mit $p = 0,26$ unterpowert. Belastbar ist: dass die Baseline keinen Vorsprung hat, dass `nstep 10` das Bündel erklärt, dass die zwei übrigen Faktoren nichts tun, und dass `nstep 10 + Bootstrap` die Baseline signifikant schlägt. Nicht belastbar ist die Behauptung, der Bootstrap-Zusatz sei *für sich* signifikant.

Nur fünf verschiedene synthetische Pfade. Bei 20 Agenten und fünf Bootstrap-Replikaten teilen sich je vier Seeds dieselbe synthetische Reihe. Die Seeds variieren dann nur die Exploration, nicht die Umgebung. Ein Lauf mit einem eigenen Pfad pro Agent wäre der nächste saubere Schritt.

Die In-Sample-Marge der Bootstrap-Arme ist nicht vergleichbar. Sie wird auf dem echten Fenster gemessen, auf dem diese Agenten nie trainiert haben; die Zeile ist deshalb nur als Overfitting-Indikator lesbar, nicht als Leistungsmaß.

Daten und Umgebung. Es gelten die Einschränkungen aus Abschnitt 4: Beobachtungsvektor 334 statt 205, lokale CSVs statt `yfinance`. A3C nutzt das gleiche Environment wie A2C, die Abweichungen wirken also auf beide Arme gleichermaßen.

5.7 Fazit des A3C-Teils

Die Baseline bestätigt den A2C-Befund auf einem zweiten Algorithmus: kein belegbarer Vorsprung vor Buy-and-Hold. Die Zerlegung zeigt, dass der Effekt der naheliegenden “Regularisierung” in Wahrheit aus der **Kredit-Zuweisung** stammt (`nstep` 5 $\rightarrow$ 10); Entropie-Bonus und Beobachtungsrauschen sind in dieser Größenordnung wirkungslos. Der **Block-Bootstrap** liefert als zweiter Hebel den besten Testwert und den ersten signifikanten Vorsprung gegenüber der Baseline — allerdings erst in der Kombination. Konzeptionell ist das der interessantere Befund: Der Agent braucht den realen historischen Pfad nicht, um auf dem Testfenster gleichauf oder besser zu sein; viele plausible Pfade genügen.

6 Artefakte und Reproduktion

Tabelle 10: Artefakte des Updates.

Datei	Inhalt
AIF_Update_2026.pdf	dieser Report (aus LaTeX gebaut)
latex-update2026/	LaTeX-Quelle des Reports (Tectonic)
UPDATE_2026_REPORT.md	Markdown-Fassung desselben Inhalts
figs/update_2026_a2c.png	End-NAV je Episode, Training vs. Test (Boeing, 5 Seeds)
figs/update_2026_margins.png	In-Sample- vs. Out-of-Sample-Marge je Aktie (20 Seeds)
figs/update_2026_a3c.png	A3C: Ein-Faktor-Zerlegung und Block-Bootstrap (Boeing, 20 Seeds)
figs/update_2026_1y.png	Kursentwicklung der drei Aktien, letzte zwölf Monate
updated_metrics_final.json	aktualisierte Fundamentalkennzahlen
results_ba_newdata.json	Roh-Ergebnisse Boeing (20 Seeds, Training + Test)
results_air_newdata.json	Roh-Ergebnisse Airbus (20 Seeds, Training + Test)
results_tm_newdata.json	Roh-Ergebnisse Toyota (20 Seeds, Training + Test)
results_ba_a3c*.json	A3C-Roh-Ergebnisse Boeing: Baseline, Ein-Faktor-Arme (<code>nstep10</code> , <code>ent02</code> , <code>noise0</code>)
agent_{ba,air,tm}_1..20.zip	die 60 neu trainierten A2C-Modelle
env_runner.py	Environment-Aufsatz (lokale Daten, Seriell-Patch)
train_eval_multi.py	Training und Auswertung für beliebige Ticker, mit Resume
analyze_seeds.py	Auswertung: Mittel, Streuung, 95%-KI, gepaarter t -Test
plot_margins.py	Abbildung Train- vs. Testmarge je Aktie
train_eval_a3c.py	A3C-Training und -Auswertung (asynchron, n -Schritt>Returns, Block-Bootstrap)
plot_a3c_ablation.py	Abbildung A3C-Ein-Faktor-Zerlegung und Bootstrap
train_eval.py	Training und Auswertung (Notebook-Protokoll, 5 Agenten)
{BA,AIR,TM}_5Y_daily.csv	Tagesdaten der drei Aktien (fünf Jahre)

Reproduzieren.

```
cd aif-latex
python3 aif_run/train_eval.py                # Boeing, 5 Agenten, Notebook-Protokoll (~35 min)
python3 aif_run/train_eval_multi.py BA      # 20 Seeds, Boeing
sh aif_run/run_seeds20.sh                   # 20 Seeds fuer TM und BA
python3 aif_run/analyze_seeds.py TM BA AIR  # Tabellen + Signifikanztests
python3 aif_run/plot_margins.py             # Abbildung Train- vs. Testmarge
```

```
python3 aif_run/train_eval_a3c.py BA          # A3C auf Boeing, 20 Seeds (~2 h)
python3 aif_run/plot_a3c_ablation.py          # Abbildung A3C-Zerlegung + Bootstrap
cd paper/latex-update2026 && tectonic -X compile main.tex --outdir out  # Report-PDF
```

Ein Lauf ist unterbrechbar: `train_eval_multi.py` überspringt Agenten, die bereits als `.zip` vorliegen, und schreibt `results_<ticker>_newdata.json` inkrementell fort.

Hinweis zu den Datenquellen. Yahoo/`yfinance` ist von diesem Host hart geblockt (HTTP 429 auf allen Endpunkten, auch nach zwanzig Minuten). Boeing und Toyota stammen daher aus der History-API von `stockanalysis.com`, die Airbus-Kurse aus der von Chris nachgelieferten Yahoo-Reihe zu `AIR.PA` (Euronext Paris, in Euro). Achtung: der Endpunkt `/api/symbol/s/AIR/history` liefert *AAR Corp (US)* und nicht Airbus – die daraus zunächst abgeleiteten Airbus-Zahlen waren falsch und wurden verworfen.